

Bug.

Verificar un sistema que no tiene servidor.

Un juego de cartas donde la partida vive en los navegadores de quienes juegan, sin backend que arbitre. Nadie tiene la versión buena del estado: **o** **están todos de acuerdo, o no hay partida.**

COMPROBACIONES

226

automatizadas

FALLOS

0

última ejecución

DEFECTOS HALLADOS

17

+ 4 del laboratorio

PIPELINE

SUCCESS

7 etapas · 7,2 min

Cuatro ejes que hay que probar sin un árbitro central

EJE 1 · COMUNICACIÓN

WebSocket + WebRTC

El servidor solo presenta a dos peers. La partida entera viaja por DataChannels cifrados.

EJE 2 · SINCRONIZACIÓN

Reloj de Lamport

Orden total: los eventos concurrentes se desempatan por `peerId`, igual en todos.

EJE 3 · EXCLUSIÓN MUTUA

Testigo con secuencia

El turno es el recurso crítico. Cada nodo guarda su réplica del estado.

EJE 4 · TOLERANCIA A FALLOS

Latidos + Bully

Se detecta al caído, se elige nuevo líder y la mesa sigue jugando.

Verificar pregunta *¿está bien construido?* Validar pregunta *¿sirve para lo que se pidió?* Un sistema distribuido añade una tercera: **¿siguen todos de acuerdo?** — y esa es la que sostiene el diseño entero.

La pirámide es ancha por abajo – y también por arriba



Hay cosas que **solo** se pueden probar arriba. `RTCPeerConnection` no existe en Node, así que la malla de verdad únicamente se prueba en un navegador.

Y los defectos lo confirman: de los 17 encontrados, **11 eran invisibles para una prueba unitaria**. Aparecieron jugando, en un navegador real, o atacando el servidor a mano.

5 395 líneas analizadas como un solo proyecto

COBERTURA

89,0 %

1 250 líneas por cubrir

DUPLICACIÓN

1,3 %

umbral < 3 %

DEUDA TÉCNICA

A

751 min · 162 smells

COMPLEJIDAD

1 410

cognitiva 967

Cobertura de líneas por paquete



La interfaz no se mide con unitarias: la prueba Cypress en un navegador, que es donde han aparecido sus fallos. Cada exclusión lleva escrito su motivo.

⚠️ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)



ProjectsIssuesRulesQuality profilesQuality gatesMore

Bug - juego de carta...
Project

Analysis

Overview

Issues

Security hotspots...
Deprecated

Reporting

Measures

Activity

Quality gate history
New

Project

<> Code

Project information

Bug - juego de cartas P2P descentralizado / Overview

Overview

5.4k Lines of Code · Version 1.7.1 · Last analysis 6 minutes ago

Quality gate

✓ Passed

⚠️ The last analysis has warnings. [See details](#)

New Code

Overall Code

New Code Since August 8, 2026 Started 6 hours ago

New issues

0

Required = 0

Accepted issues

0

Valid issues that were not fixed

Coverage

100%

Required ≥ 80.0%
On 2 New Lines to cover.

Duplications

0.0%

Required ≤ 3.0%
On 376 New Lines.

Security Hotspots

0

Deprecated

A

⚠️ Limited security analysis

SonarQube 26.8 · proyecto bug-p2p · análisis lanzado por el pipeline

4.1 · Calidad de código

5

Un número que no se mira no significa nada

1 BUG · ACCESIBILIDAD

El fondo del diálogo de jugada cerraba con ratón y no con teclado. Se arregló cambiando el elemento: un `<dialog>` nativo cierra con Esc, atrapa el foco y se anuncia como diálogo.

Los dos parches evidentes no funcionaban o cambiaban una queja por otra.

22 VULNERABILIDADES · UNA SOLA REGLA

Todas son `Math.random`. Revisadas una a una: 16 son partículas de efectos, el resto códigos de sala y semillas **públicos por diseño**.

La única que merecía dudar: `uid()` genera el *secret* de reconexión. Sus dos ramas reales usan `crypto`; la de `Math.random` es inalcanzable para quien pueda jugar. 0 explotables.

Y una lección de propina: escribir los comentarios en español dispara falsos positivos de reglas pensadas para inglés — la palabra «todo» se lee como un marcador `TODO` pendiente.

Siete etapas, ordenadas por lo que falla más barato primero



La etapa 5 es la única que no tumba la construcción: un servidor de métricas apagado no debería convertir un commit sano en un commit roto. Se marca **inestable**.

 **Jenkins** / bug-p2p-rama

  [Sign in](#)

 Status

 Changes

 Git Polling Log

Builds > 

 Filter 

Today

 #1 4:01 AM 

 **bug-p2p-rama**

Monorepo de Bug. Dispara con cada commit de la rama principal (sondeo del repo montado en /repo-src).



Last Successful Artifacts

 lcov.info	41.41 KiB	 view
 distribuida-2026-08-09T04-08-59.json	6.97 KiB	 view
 distribuida-ultimo.json	6.97 KiB	 view
 seguridad-2026-08-09T04-08-33.json	7.07 KiB	 view
 seguridad-ultimo.json	7.07 KiB	 view



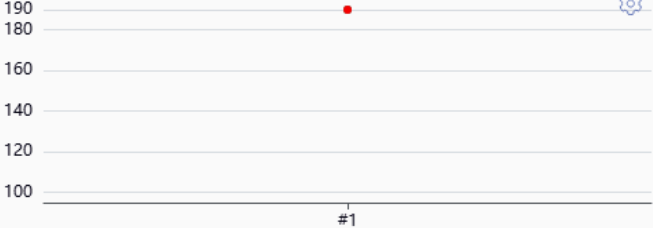
Latest Test Result (no failures)

Permalinks

- [Last build \(#1\), 26 min ago](#)
- [Last stable build \(#1\), 26 min ago](#)
- [Last successful build \(#1\), 26 min ago](#)
- [Last completed build \(#1\), 26 min ago](#)

Test Result Trend

 Passed  Skipped  Failed 



Test Result	Count
Passed	190
Skipped	0
Failed	0

REST API Jenkins 2.568.2

Las 7 etapas verdes en 7,2 min · 190 pruebas sin fallos · artefactos de cobertura, seguridad y validación distribuida

El pipeline llevaba semanas escrito. Nunca se había ejecutado.

- 1 **Jenkins no arrancaba.** El Job DSL dinámico (`scmGit { remotes }`) ya no existe en las versiones de hoy: se llevaba por delante a Configuration-as-Code y el contenedor moría en `Exited (5)`.
- 2 **La corrección no llegaba.** `casc.yaml` vivía dentro de `/var/jenkins_home`, que es un volumen: ahí manda el volumen, no la imagen. Reconstruir no cambiaba nada.
- 3 **El checkout moría.** El plugin de Git no clona rutas locales sin habilitarlo explícitamente.
- 4 **Y moría otra vez, un paso más adentro.** Git rechazaba el repo montado por pertenecer a otro usuario. El mismo error del punto 2 con otro disfraz: `--global` escribe en `$HOME...` que es el volumen.

Y uno silencioso, el peor: el token de Sonar nunca entraba en el contenedor, así que el pipeline habría saltado el análisis en cada build **con el mismo amarillo que si el laboratorio estuviera apagado**. Un fallo disfrazado de comportamiento tolerado no lo investiga nadie.

18 pruebas funcionales, 4 de ellas con tres nodos reales

MENU.CY.TS

8

Flujos de entrada, invitación por QR, validación del enlace, móvil de 360 px

PARTIDA-LOCAL.CY.TS

6

Reparto, robar, no pasar sin robar, rotación del turno

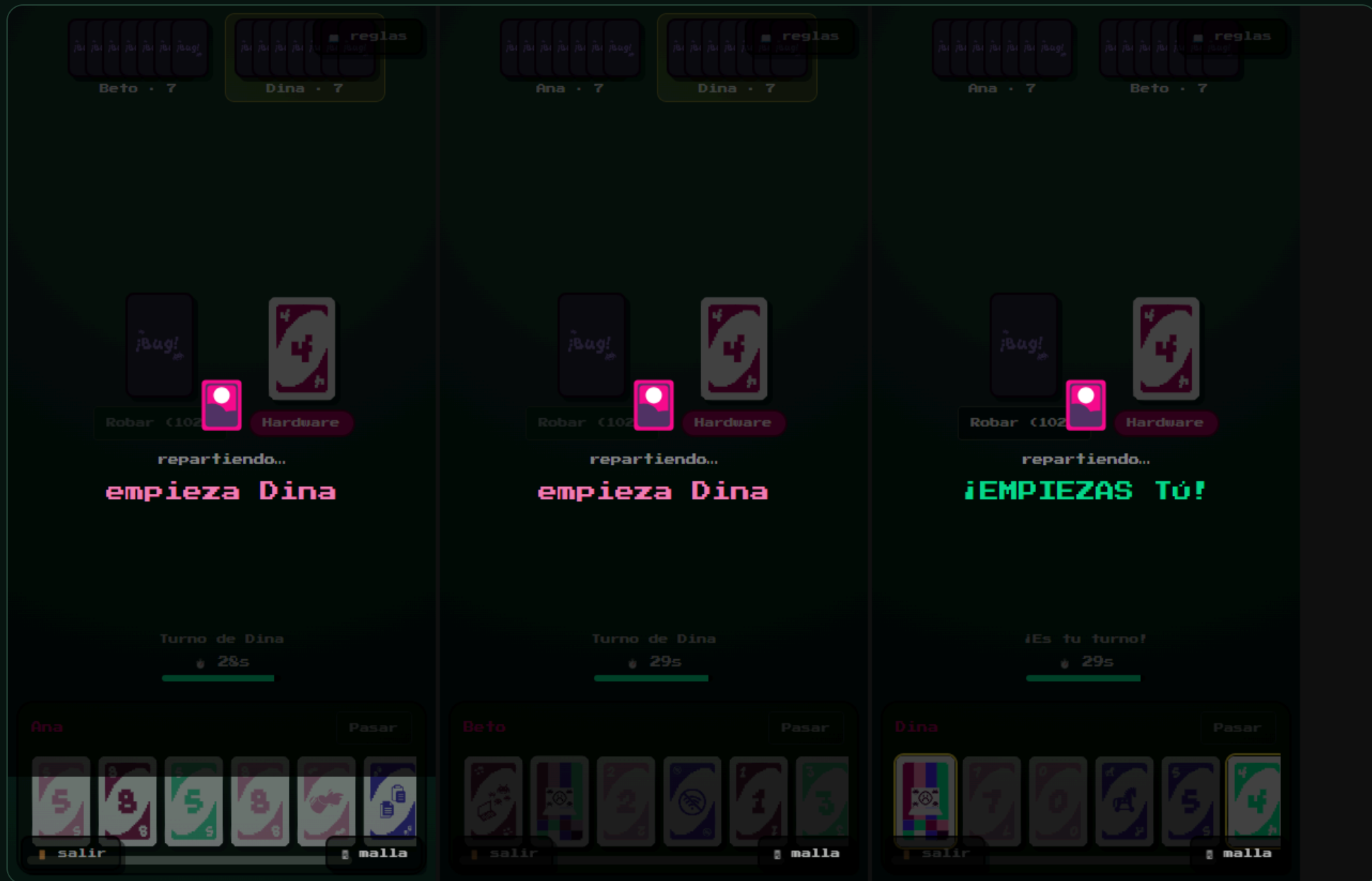
DISTRIBUIDO/MALLA.CY.TS

4

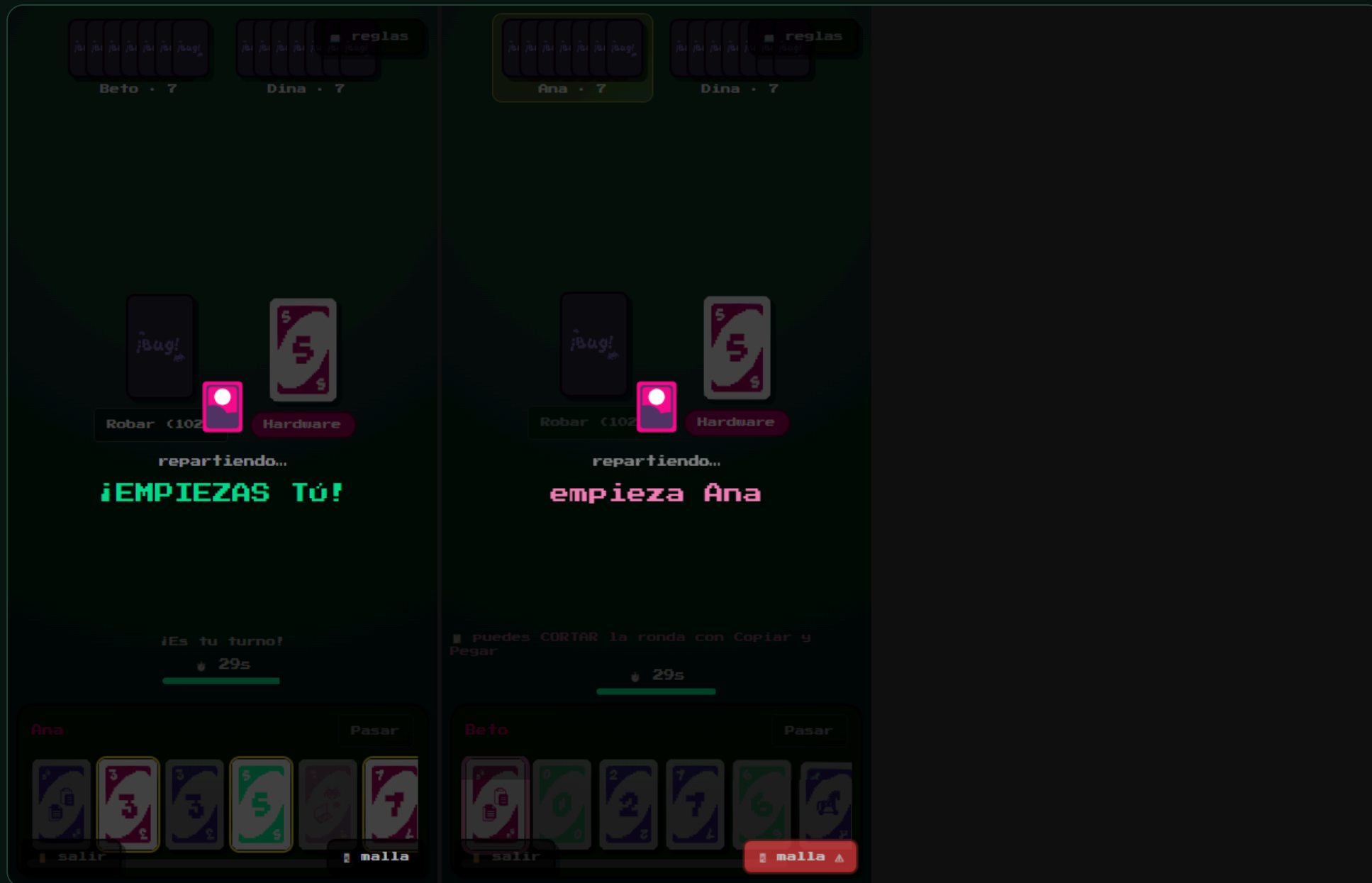
Tres nodos simultáneos con WebRTC real, comparando la huella del estado

Los tres nodos son contextos de navegación independientes, cada uno con su propia `RTCPeerConnection`. La prueba no mira cartas en pantalla: compara **la huella del estado replicado** de los tres — el mismo número que se proyecta en la Pantalla Maestra.

Que el lobby lo muestre el anfitrión no bastaría, y por eso se comprueba en los tres: el que llega segundo se conecta con el tercero *por la malla*, no por el servidor.



Ana, Beto y Dina · tres contextos de navegación, mismo pozo y mismo turno · la prueba compara sus tres huellas de estado



Se quita el tercer nodo de golpe · los dos que quedan siguen convergiendo entre ellos

6 vulnerabilidades encontradas. Las 6, corregidas y vigiladas.

SEVERIDAD	ATAQUE	ANTES	AHORA
Crítica	S3 · secuestro de plaza	×	✓
Crítica	S8 · malformados	×	✓
Alta	S1 · suplantación del emisor	×	✓
Alta	S2 · inyección desde fuera	×	✓
Media	S6 · agotar conexiones	×	✓
Media	S7 · mensaje de 32 MB	×	✓
S4 · S5 · S9 · S10 · S11	ya defendidos	✓	✓

Burp es con lo que se **encuentran** los fallos; no con lo que se **vigilan**. Un hallazgo que solo existe en la memoria de quien lo encontró vuelve a aparecer en tres commits.

Por eso los once están escritos en [attack-suite.mjs](#) y corren en cada construcción contra el servidor real.

Y el resultado que más dice del diseño no es ningún ataque bloqueado, sino lo que **no aparece** en el proxy: ni una carta. El juego no pasa por ahí.

Cuarenta caracteres bastaban para dejar la feria sin señalización

S8 · MENSAJES MALFORMADOS

```
{"t":"introduce","room":"AB12",  
  "peerId":"x","tried":7}
```

Enviado desde la consola del navegador por cualquier jugador.

El más instructivo, en cambio, es S1: el cifrado de WebRTC protege el contenido del canal, no la identidad de quien lo abrió. Esa la garantiza la señalización, o no la garantiza nadie.

El servidor hacía `new Set([peerId, ...msg.tried])`. Desparramar un número lanza una excepción, y una excepción dentro de un manejador de eventos de Node sube hasta `uncaughtException` y **mata el proceso**.

Corregido en dos capas: `parseClientMsg` valida tipo, longitud y forma de todo lo que entra, y el manejador va envuelto en un `try/catch` para el fallo que no se nos ocurrió.

No un verde: una medición

ID	PROPIEDAD	LO QUE SE MIDIÓ
D1	Consistencia	3/5/10 nodos, latencias de 5–120 ms y 64 entregas duplicadas → todas las réplicas con la misma huella
D2	Orden causal	12 órdenes de entrega distintos → una sola secuencia acordada
D3	Exclusión mutua	200 cesiones del testigo · 0 violaciones · 234 anuncios rezagados descartados
D4	Detección de fallos	sospechoso a 2 500 ms , caído a 6 000 ms , readmitido al volver
D5	Elección de líder	Bully con 3/5/10 nodos y caída encadenada del favorito → acuerdo unánime
D6	Recuperación	el rezagado converge al completar el log; el corrupto no — necesita adoptar
D7	Rendimiento	factor de replay 18,4× / 6,5× / 8,5× · la malla usa 240 mensajes donde un servidor usaría 360

D6 justifica una decisión de diseño: a un nodo corrupto no le basta con recibir los eventos que le faltan, porque además tiene eventos de más. Por eso la reparación tira el log propio y adopta el ajeno.

Un criterio de aceptación también puede estar mal

D7 exigía que el caso realista convergiera en **menos de 2 segundos de reloj de pared**. Con la máquina cargada, falló — *y las tres réplicas habían convergido perfectamente*.

Un umbral que depende de lo que haya de fondo produce rojos sin defecto detrás. Y un rojo que no significa nada **enseña a ignorar los demás**.

Lo primero que hay que poder creer es la medida. Encontrar esto es tan parte de la V&V como encontrar un bug en el producto — y llega antes.

ANTES · MIDE LA MÁQUINA

`msRealista < 2000`

AHORA · MIDE EL ALGORITMO

`aplicaciones por evento ≤ 30×`

Sale idéntico en un portátil cargado y en el agente de CI. Los tiempos se siguen midiendo; ya no deciden.

Solo 6 los habría visto una prueba unitaria



CICLO DE VIDA DE REACT

La sala no se creaba: StrictMode monta, desmonta y vuelve a montar.

ANCHURA EN CSS

El menú desbordaba a 360 px — y así entra casi todo el mundo en la feria.

UN EVENTO A DESTIEMPO

Llegaba antes de existir el motor y se tiraba: divergencia permanente.

No es un argumento contra las pruebas unitarias — son 190 y sostienen el motor y los algoritmos. Es un argumento contra leerlas como si fueran la V&V entera.

Lo que este proceso **no** cubre

- ! **Partida entre redes distintas de verdad.** El código lleva STUN y TURN configurable, pero la prueba necesita dos ubicaciones y no se ha hecho. Es el riesgo abierto más importante para la feria.
- **Diez jugadores en móviles reales.** Está medido en simulación (D1, D7), no con diez teléfonos.
- **Accesibilidad completa.** No es requisito del enunciado y no se ha auditado — más allá del diálogo que salió del análisis estático.
- **Cypress no corre en cada commit.** El agente no tiene navegador; las 18 funcionales se lanzan a mano.

Un plan de V&V que promete cubrirlo todo no es creíble. Decir dónde no se ha mirado es parte del informe, no una concesión.

Un requisito sin prueba es una intención.

Y una configuración que nunca se ha ejecutado, también. El pipeline llevaba semanas escrito y no arrancaba; el criterio de D7 medía la máquina en vez del algoritmo; el arnés de tres nodos perdía una carrera. Nada de eso se ve leyendo el código.

SONARQUBE

89,0 %

cobertura · dup. 1,3 %

JENKINS

7/7

etapas en verde

CYPRESS

18/18

3 nodos, WebRTC real

SEGURIDAD · DISTRIBUIDA

11/11 · 7/7

ataques · propiedades